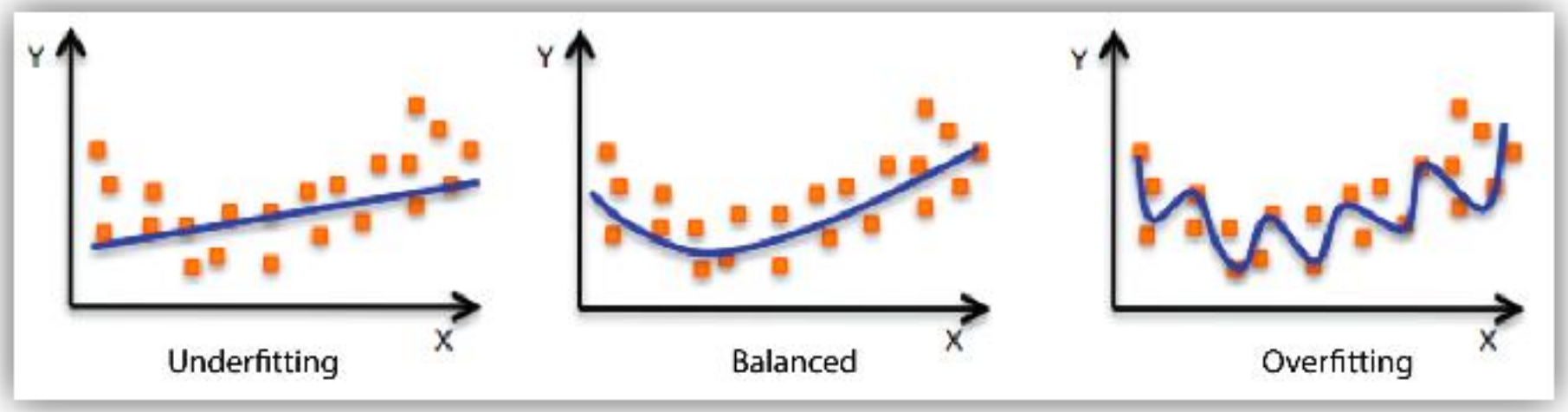
# ML Evaluation

# Topics

1. Overfitting and Underfitting
2. Bias, Variance and Bias-Variance Tradeoff
3. Train, Test vs Validation Dataset
4. Cross-Validation Techniques
    a. K-Fold
    b. Stratified K-Fold
    c. LOOCV
5. Different ML Evaluation Metrics
    a. Accuracy
    b. Confusion Matrix - Precision, Recall, F1 Score
    c. Confidence

# Overfitting and Underfitting

**Overfitting** and **Underfitting** are two crucial concepts in machine learning and are the prevalent ***causes for the poor performance*** of a machine learning model.

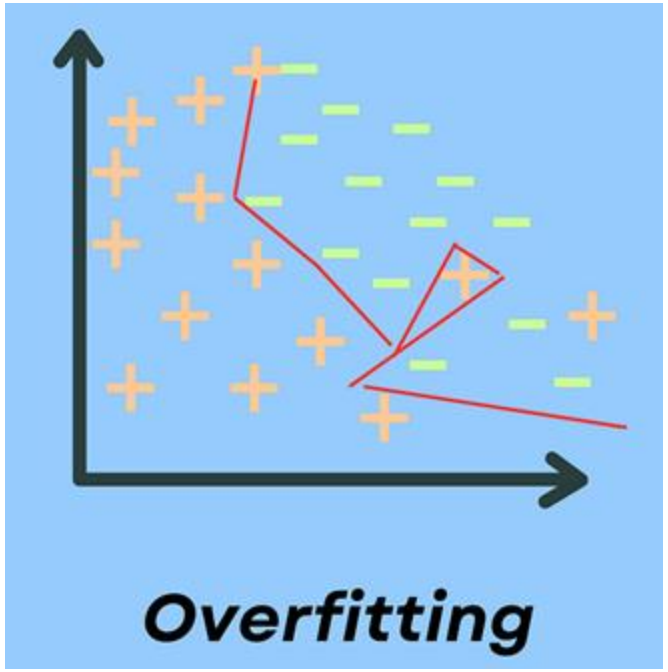


# Overfitting in Machine Learning

**Overfitting occurs when a model performs exceptionally well on training data but poorly on unseen test data.**

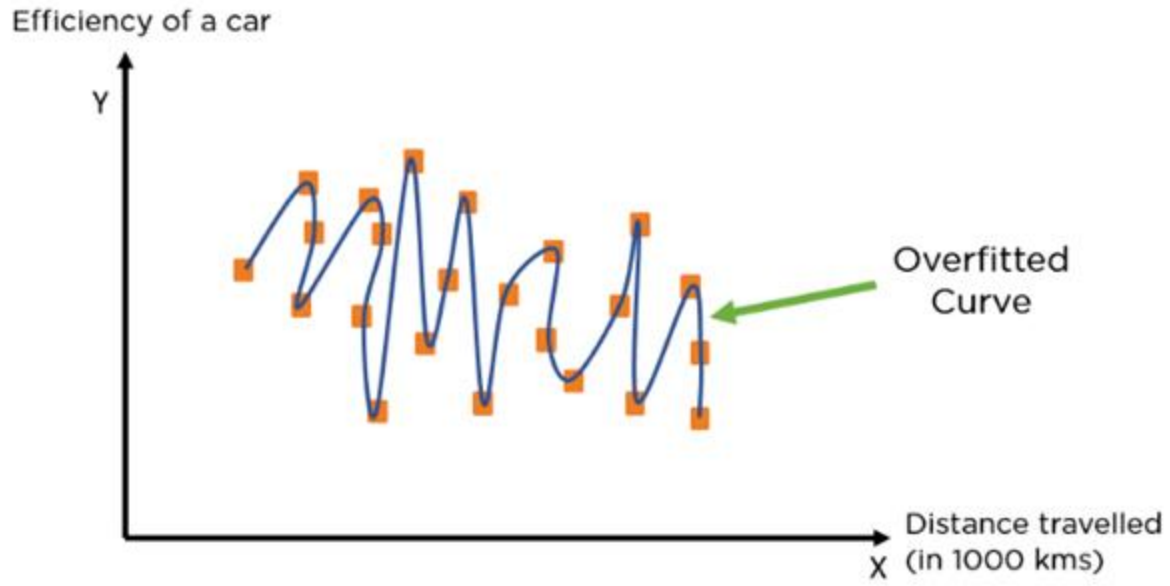
**Key Points:**

- **Cause:** The model learns noise and random fluctuations in the training data too well, rather than the underlying patterns.
- **Result:** An overly complex model that fails to generalize to new data.
- **Impact:** Decreased performance on test data, undermining the model's predictive capability.



# Causes of Overfitting

- **Noisy Data:** Irrelevant information in training data leads to learning noise instead of patterns.
- **Model Complexity:** Overly complex models memorize noise instead of generalizing.
- **Insufficient Data Size:** Small or non-diverse datasets lead to memorization of specific examples.



# Some ways to Tackle Overfitting

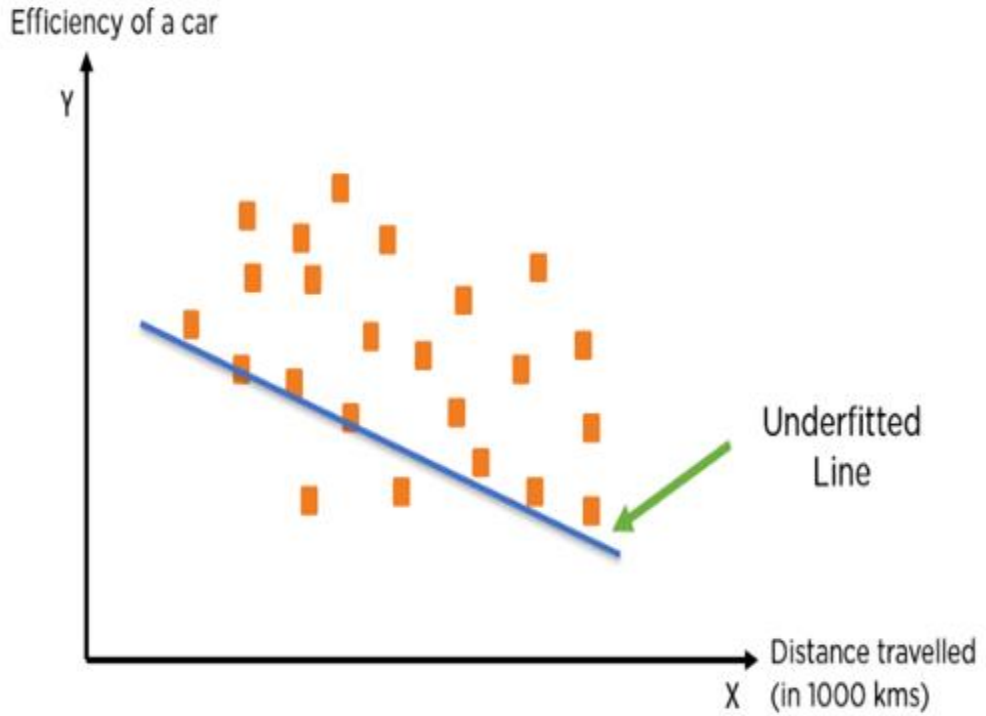
- **Training model with sufficient data**
  - Add more data: More diverse and representative data leads to more robust and generalizable models.
- **Using K-fold cross-validation (splitting train-validation-test) → Next Topic**
  - Helps evaluate model performance and promote generalization to new data.
- **Using Regularization techniques (later topic)**
  - Reduce overfitting by adding penalties to model parameters, making them less sensitive to noise

# Underfitting

**Our model is too general!** It memorized one rule and is applying it everywhere.

- Think about college admission process only looking at GPA

**Definition:** when a model fails to learn the patterns in the training data effectively, resulting in poor performance on both the training data and new, unseen test data. Tt is known as underfitting.



# Causes of Underfitting?

- **Too Simple:** The model is too simplistic to capture data complexity.
- **Missed Features:** Important details are overlooked, weakening data representation.
- **Unreliable Predictions:** Poor training performance leads to unreliable predictions on new data.

One of the way to tackle underfitting is **Increase Model Complexity:** If the model is too simple, increasing its complexity can alleviate underfitting. E.g.

- Add More Layers or Neurons in Neural Network
- In polynomial regression, you can increase the degree of the polynomial to allow the model to fit more complex curves to the data.

# The Bias/Variance Tradeoff

A fundamental concept in machine learning that deals with the problems of overfitting and underfitting

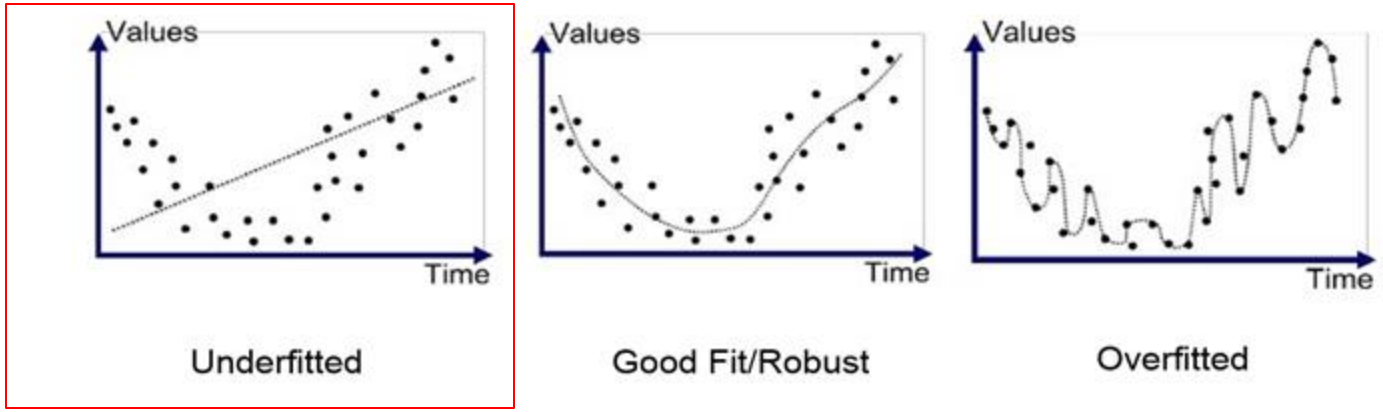
# Concept of Bias-Variance Tradeoff

As we want to minimize prediction error on both training and validation datasets, we need to know **two types of errors** in model performance-

1. Bias and
2. Variance.

# Bias

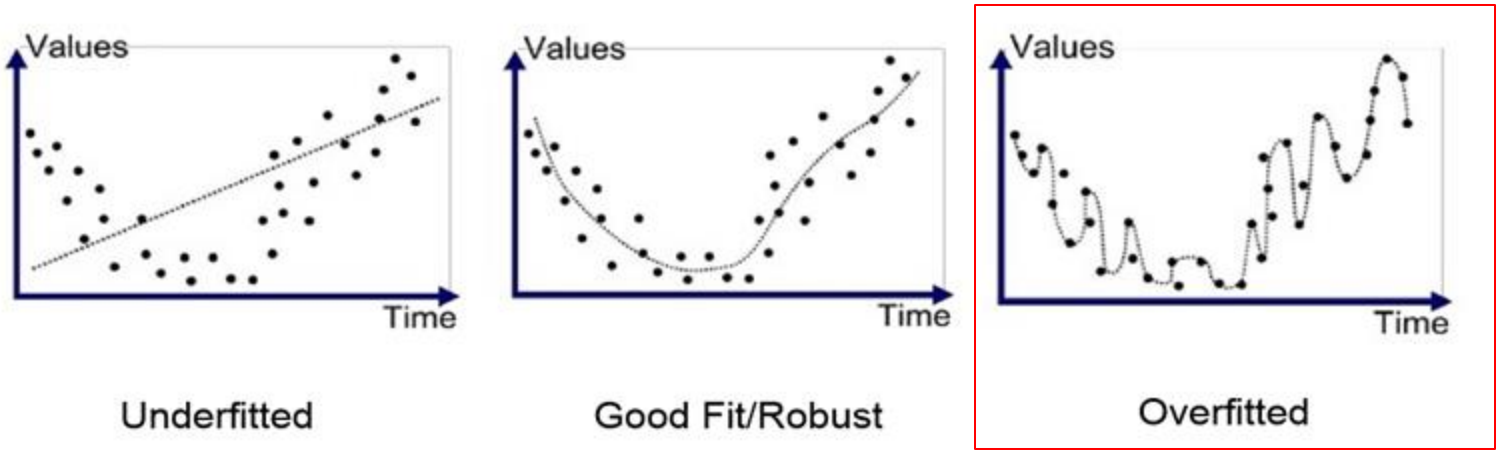
**Bias refers to the error introduced by approximating a real-world problem (which may be complex) by a simplified model.**




Bias quantifies how much the predicted values differ from the actual (expected) values.
**High bias → simple model → poor performance.**

# Variance

Variance is the error introduced when a model is overly sensitive to fluctuations in the training data (measures how much the predictions for a given data point can change when the model is trained on different subsets of the data.).

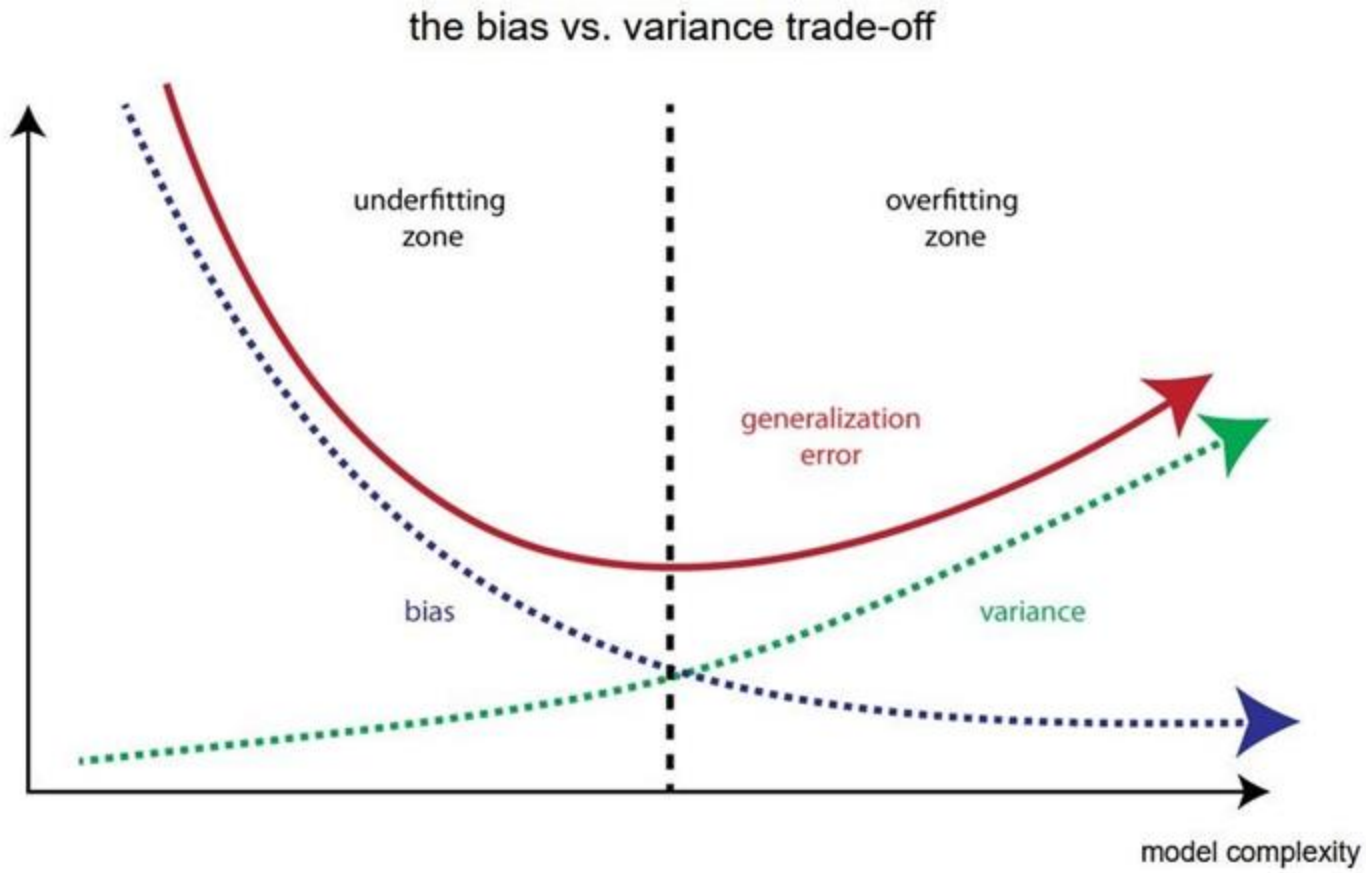



Variance is the variability of model prediction for a given data point or a value which tells us spread of our data.

**High variance → complex model → great performance on training data but poor on new data.**

# Bias-Variance Tradeoff

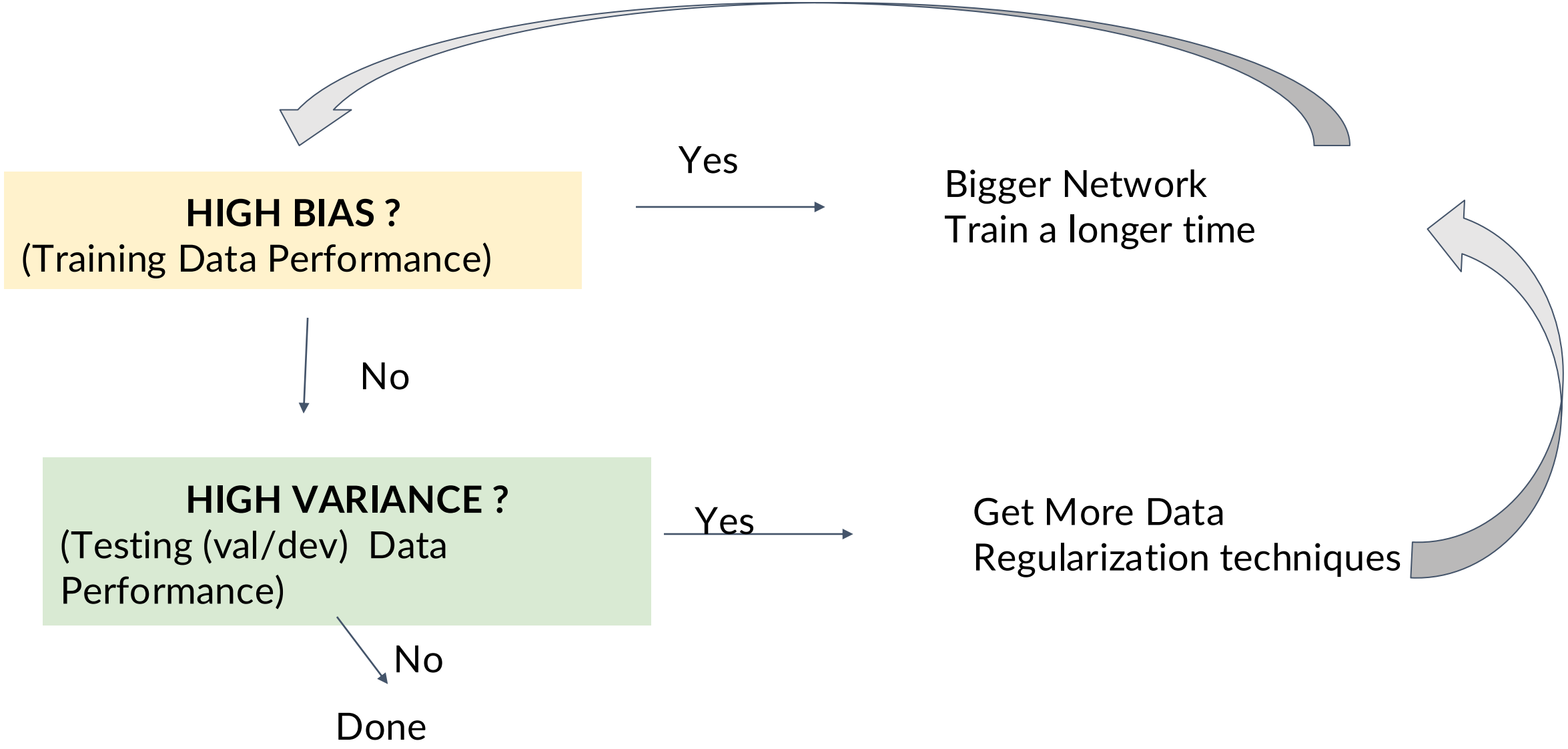
Aim for a model that minimizes both bias and variance for optimal performance.

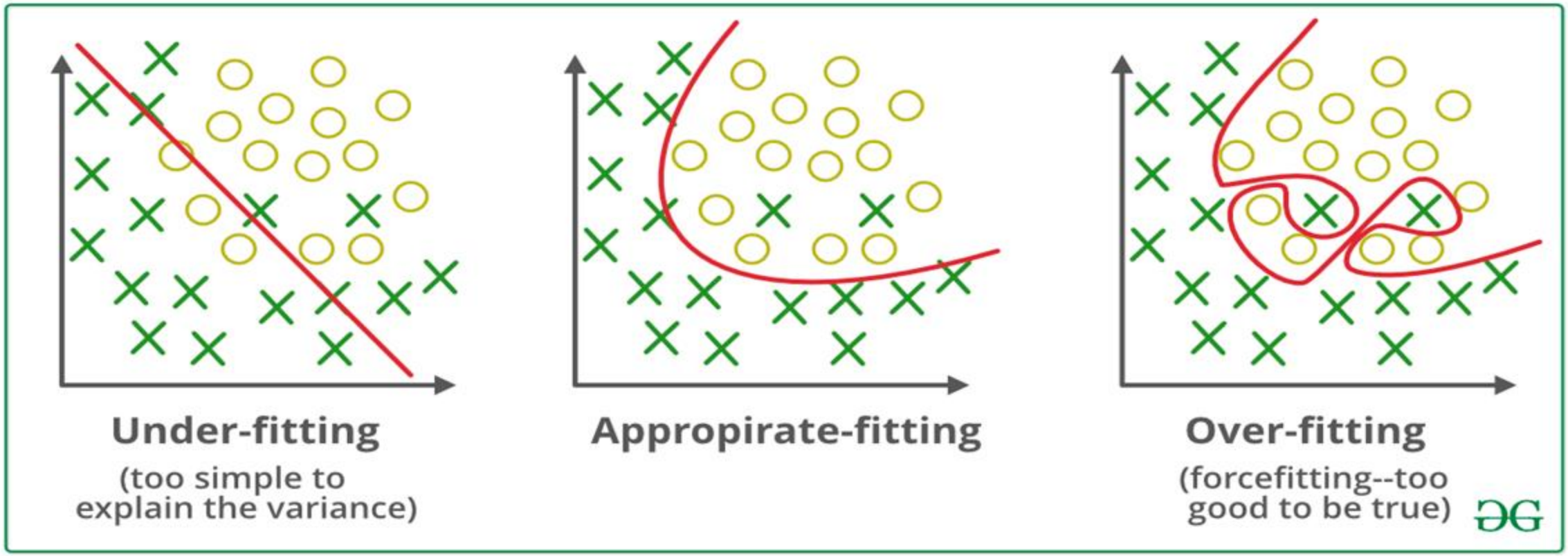


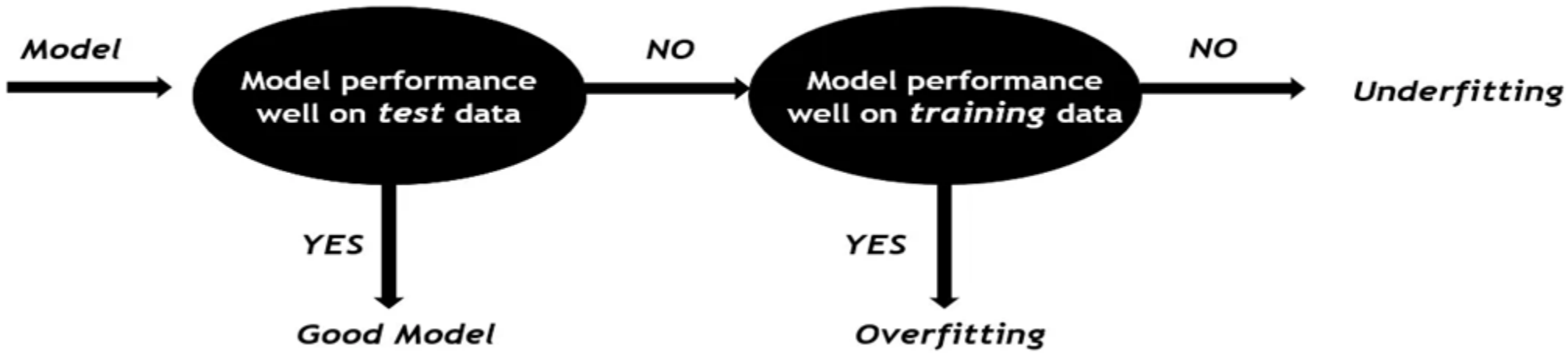

Generalization Error=Bias^2  + Variance + Irreducible Error

Irreducible Error: This is the noise inherent in the data itself, which can’t be eliminated regardless of the model.

# Basic Recipe of ML to tackle this




Under-fitting
(too simple to explain the variance)
Appropirate-fitting
Over-fitting
(forcefitting--too good to be true)


Model
Model performance well on test data
NO
Model performance well on training data
NO
Underfitting
YES
Good Model
YES
Overfitting

# Overfitting vs Underfitting

| | Overfitting | Underfitting |
|---|---|---|
| **Definition** | Model captures noise, fails to generalize. | Model is too simplistic, lacks generalization. |
| **Causes** | Too many parameters, limited data, excessive complexity. | Simple model, insufficient features, under-training. |
| **Training Performance** | Performs exceptionally well on training data. | Performs poorly on both training and new data. |
| **Generalization** | Poor generalization to new, unseen data. | Limited ability to capture underlying patterns. |
| **Typical Solutions** | Reduce model complexity, more training data, regularization. | Increase model complexity, feature engineering, more data. |

# Prevention Training Failures

# Start with “Testing and Training” (Holdout Method)

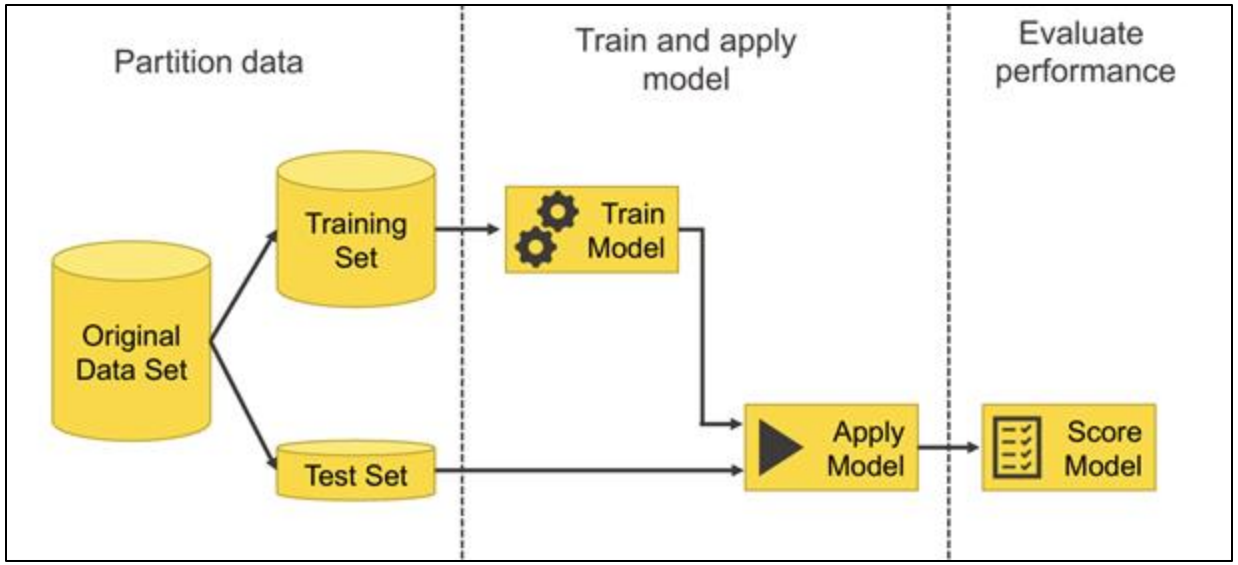
# Mitigating Overfitting: The Importance of Data Splitting in Model Training

**Issue: Training on the entire dataset can lead to overfitting, resulting in poor performance on new data.**

To compensate, sample data is often **split into three subsets**:

| | |
|---|---|
| **1. Training data (60-80%)** | Used to fit a model. |
| **2. Validation Data (10-20%)** | Evaluate model performance and aids in parameter tuning and feature selection. |
| **3. Test Data (10-20%)** | Assesses final model performance and facilitates model comparison |

# How do we know if it worked?




# We Test Our Model!

| Name | Income | Wealth | Repaid |
|---|---|---|---|
| Alice | 100,000 | 300,000 | Yes |
| Bob | 150,000 | 10,000 | No |
| Eve | 500,000 | 1,500,000 | Yes |
| Max | 100,000 | -100,000 | No |
| Joan | 100,000 | 350,000 | Yes |
| Zhi | 150,000 | 350,000 | Yes |
| Tad | 0 | 1400 | No |

Training

| Name | Income | Wealth | Repaid |
|---|---|---|---|
| Alice | 100,000 | 300,000 | Yes |
| Bob | 150,000 | 10,000 | No |
| Eve | 500,000 | 1,500,000 | Yes |
| Max | 100,000 | -100,000 | No |
| Joan | 100,000 | 350,000 | Yes |

Testing

| Name | Income | Wealth | Repaid |
|---|---|---|---|
| Zhi | 150,000 | 350,000 | Yes |
| Tad | 0 | 1400 | No |

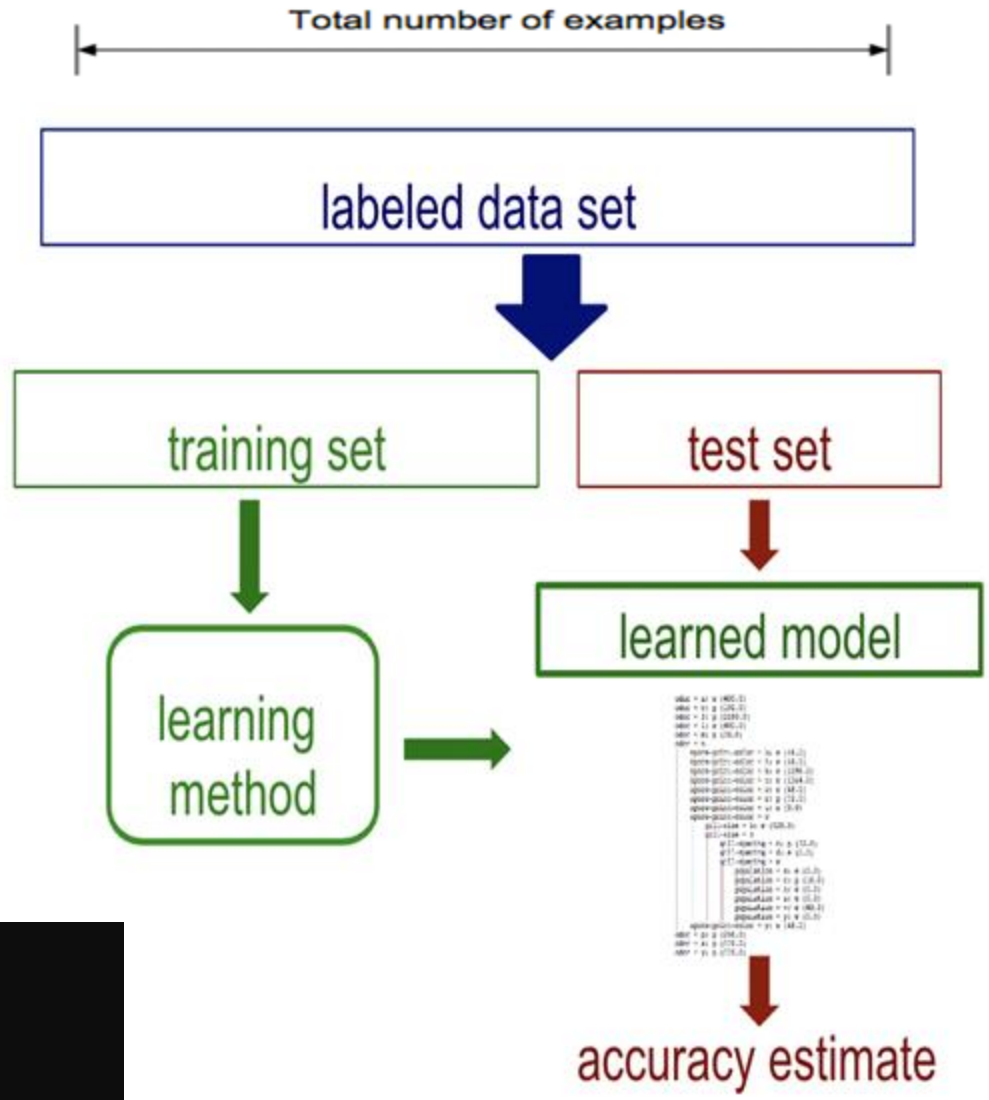
We already know their target label, use it to evaluate in on unseen data data

# Hide Some Data From Our Algorithm

# Training and Testing Data

**Training Data:** Subset of the dataset used to teach/train a machine learning model.

**Testing Data:** Subset of the dataset used to evaluate the model's performance.

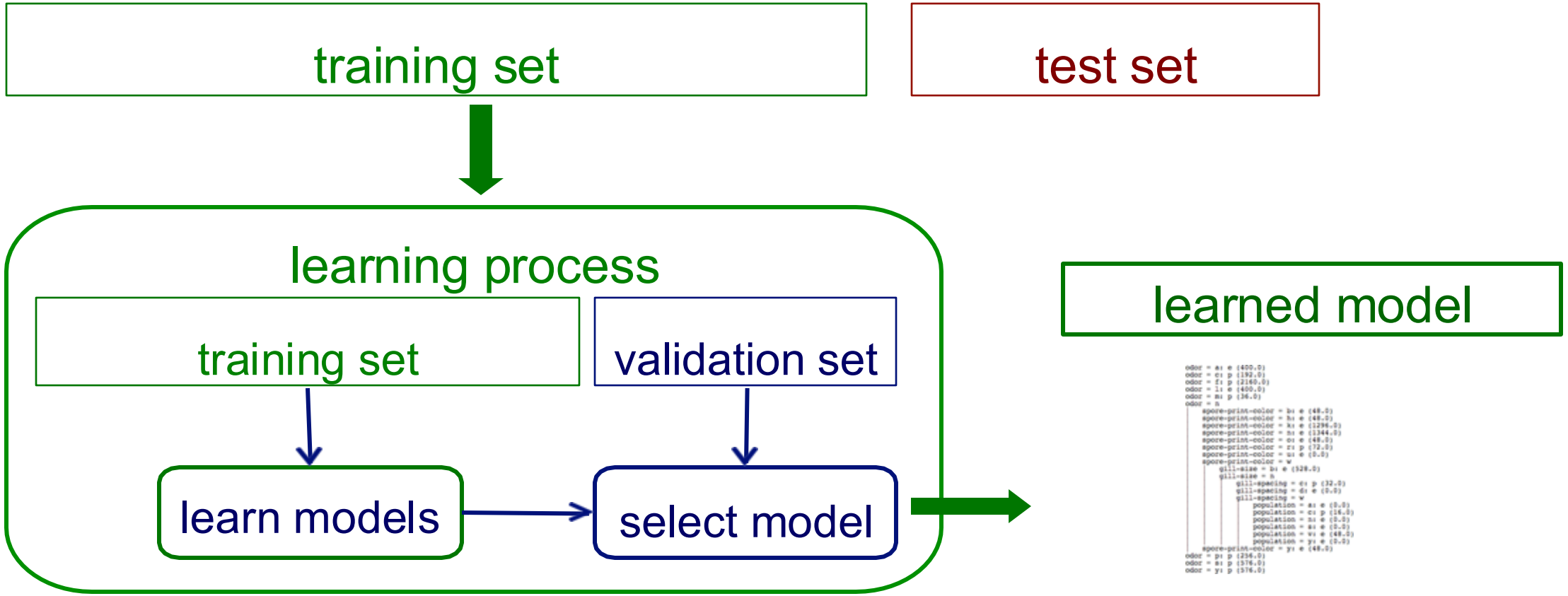



```
from sklearn.model_selection import train_test_split

# Split data into train and test sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
```

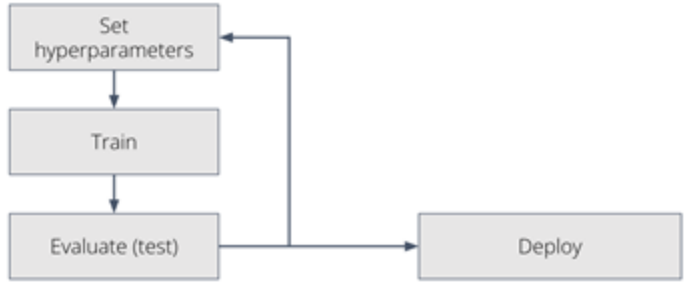
# NEXT: Validation (Tuning or Development) sets

Suppose we want unbiased estimates of accuracy during the learning process (e.g. to choose the best level of decision-tree pruning)?




Partition training data into separate training/validation sets

Hyperparameters are the settings of an algorithm that can be adjusted to optimize performance



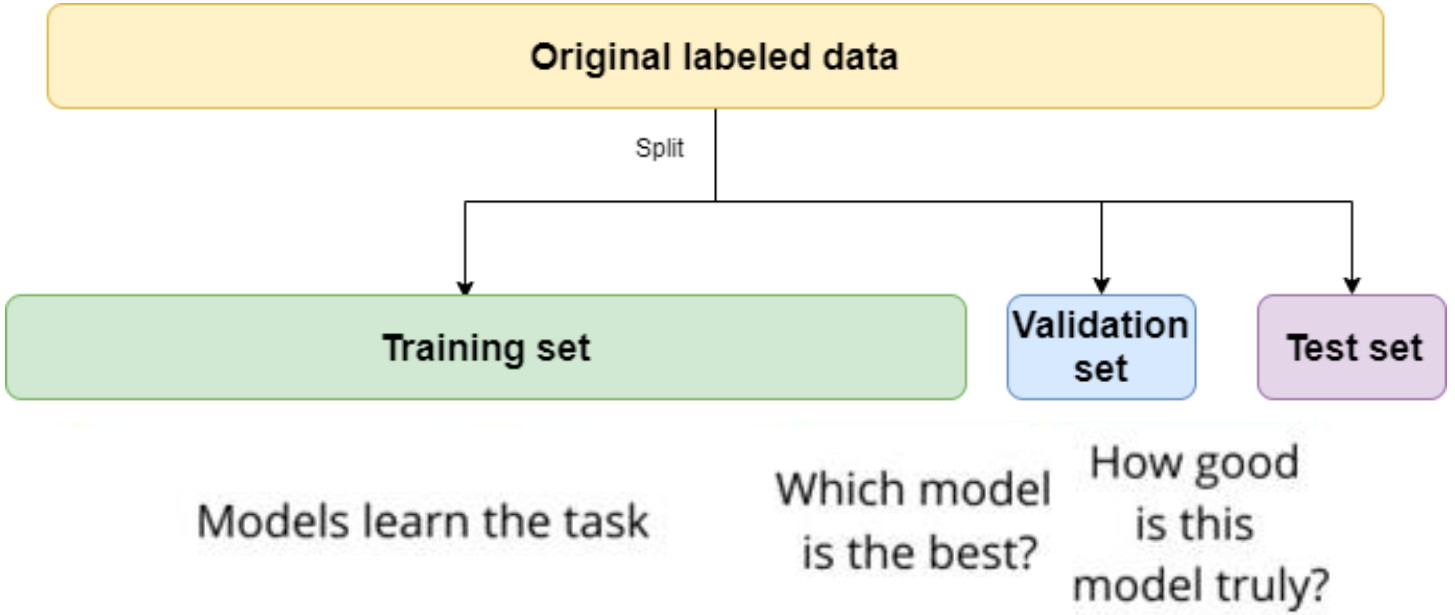

# Validation Sets

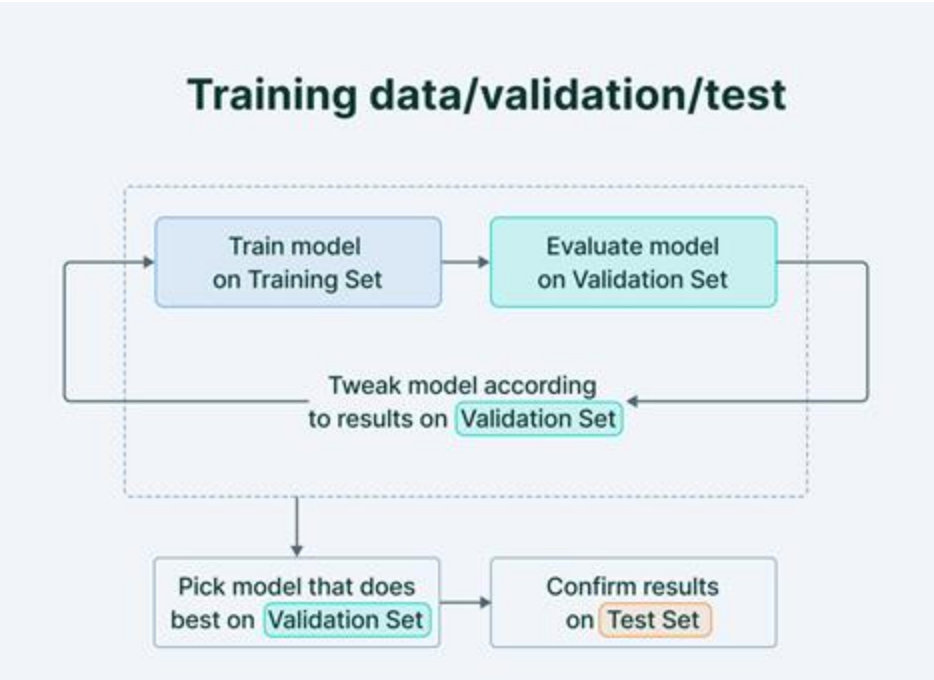
Unlike the test dataset, the validation dataset is typically **a subset of the training data** and is used iteratively during model development

- **Primary purpose:** helps evaluate the model's performance during the training phase.
- **Not used for training;** assists in fine-tuning hyperparameters and model architecture to optimize performance and reduce overfitting.
- **Iterative Use:** Used throughout training; unlike the test dataset, which is reserved for final evaluation.

# More Split : Train and Validation and Test

In machine learning, both the test and validation datasets are used to evaluate the performance of a trained model **but purpose are different!**




## How to split a dataset

**TRAINING SET**

The subset of data used to train a machine learning model

**TEST SET**

The subset of data used to evaluate the performance of a trained machine learning model on unseen examples, simulating real-world data

**VALIDATION SET**

The intermediary subset of data used during the model development process to fine-tune hyperparameters

# Limitations

**While ensuring Robust Model Evaluation**

## “Train/Validation/ Test split" method in ML can be risky!

**Non-random splits can lead to overfitting.**

E.g. one part of the data only includes people from a specific state, employees with a certain income level, or only women, it can make the model learn too much about these specific cases.

**To prevent this issue, we use "cross-validation"**

NEXT

# The Cross Validation Techniques

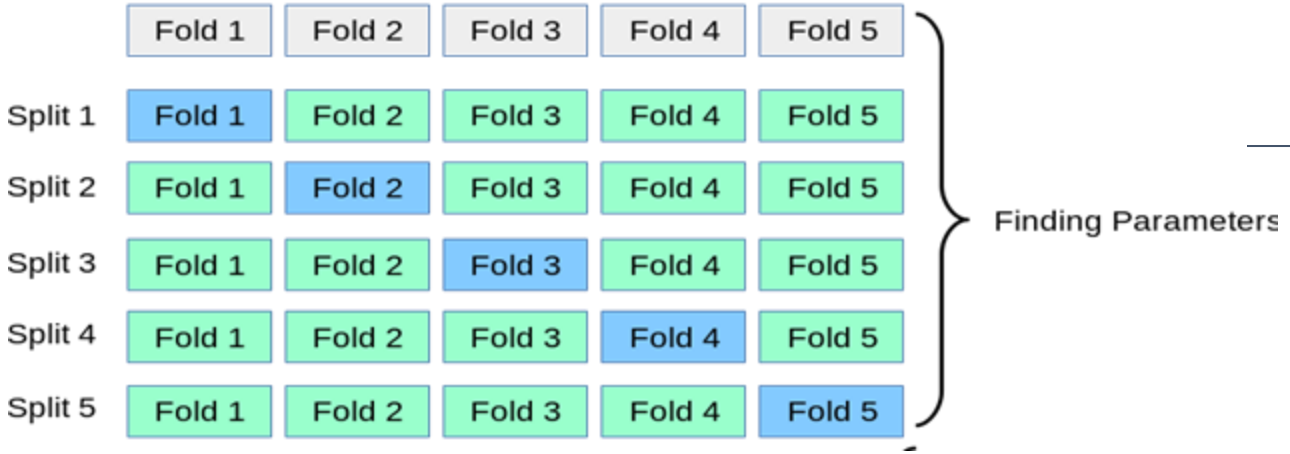
A technique for evaluating machine learning model performance by **partitioning data into multiple subsets.**

# Cross Validation Technique

**Instead of relying on a single validation dataset, cross-validation involves splitting the dataset into multiple parts to ensure that the model is learn and evaluated on different parts of the data (promoting a more balanced evaluation and improving generalization).**




**Dataset Splitting:** Data is divided into train and test.



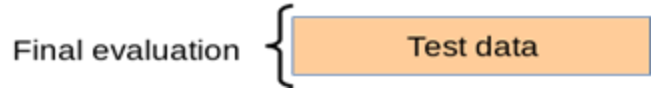

**Training and Validation: Cross Validation**

**Dataset Splitting:** Now training data is divided into subsets, called "folds".

- **Training Phrae:** In each iteration, the model is trained on all folds except one.
- **Validation Phase:** The remaining fold serves as the validation set.

**Repetition:** This process repeats for each fold as the validation set.

**Performance Assessment:** After all iterations, performance metrics are averaged to evaluate the model across all folds, ensuring good generalization and reducing overfitting.



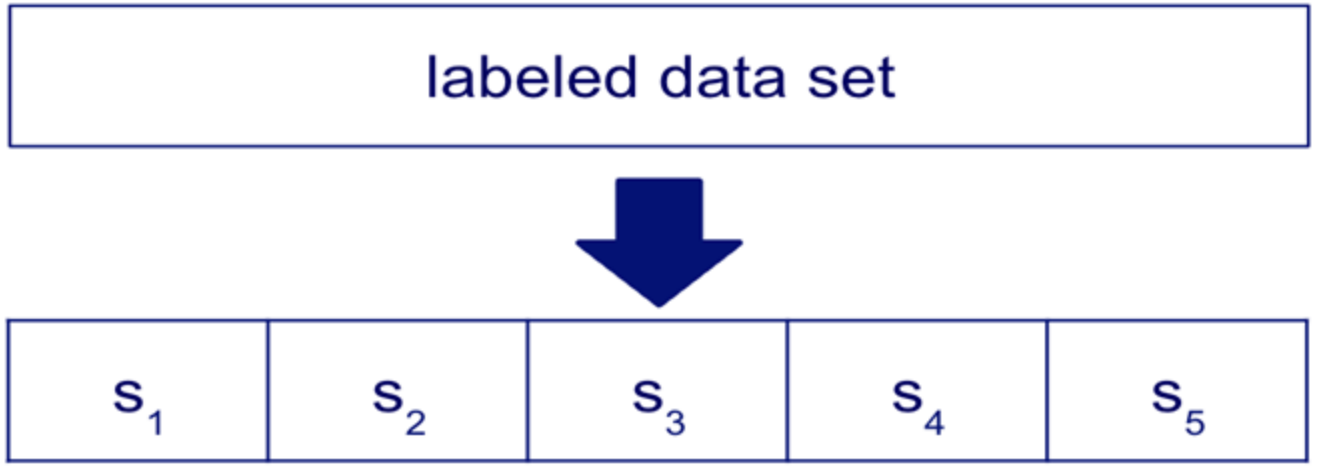

**Separate Testing Dataset Final Evaluation**: After cross-validation, a separate testing dataset (not used in training or validation) is used to evaluate the final model. This ensures an unbiased assessment of the model's performance on entirely unseen data.

Cross Validation Process

# Example: Cross validation

Suppose we have 100 instances, and we want to estimate accuracy with cross validation.

partition data into *n* subsamples




iteratively leave one subsample out for the test set, train on the rest

| iteration | train on | test on | correct |
|---|---|---|---|
| 1 | $s_2$ $s_3$ $s_4$ $s_5$ | $s_1$ | 11 / 20 |
| 2 | $s_1$ $s_3$ $s_4$ $s_5$ | $s_2$ | 17 / 20 |
| 3 | $s_1$ $s_2$ $s_4$ $s_5$ | $s_3$ | 16 / 20 |
| 4 | $s_1$ $s_2$ $s_3$ $s_5$ | $s_4$ | 13 / 20 |
| 5 | $s_1$ $s_2$ $s_3$ $s_4$ | $s_5$ | 16 / 20 |

For validation

accuracy = 73/100 = 73%

# Types of Cross Validation Techniques

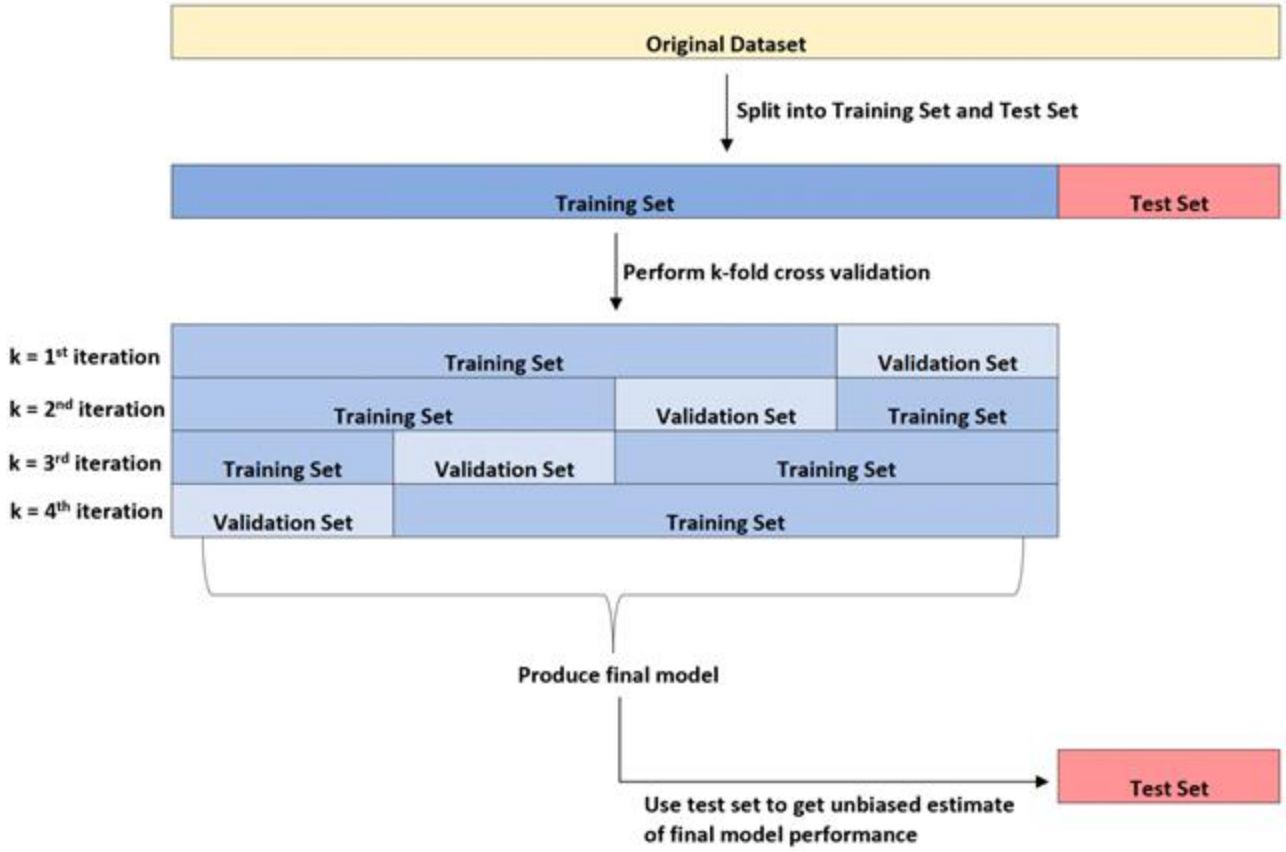
- K-Fold Cross-Validation
- Leave-One-Out Cross-Validation (LOOCV)
- Many more…..

# K-Fold Cross Validation

**A technique to evaluate a machine learning model by dividing the dataset into K equal-sized subsets, or "folds."**

**How It Works:**

- **Data Partitioning:** The dataset is randomly split into K folds.
- **Training and Validation:** The model is trained on K-1 folds and validated on the remaining one fold. This repeats K times, with each fold serving as the validation set once.
- **Performance Aggregation:** After all iterations, average the performance metrics from each fold for a comprehensive evaluation.



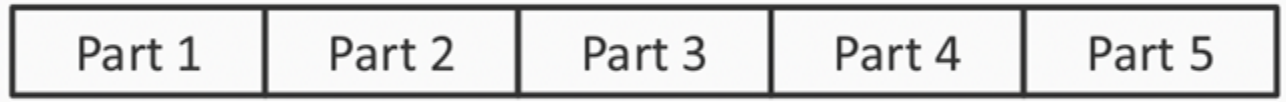

Example of Training and Validation (Cross Validation K=4 Fold in this case) and Testing Dataset

**Common Values for K:** Typical values are 5 or 10, adjustable based on dataset size.

# K-fold cross-validation

*Given a particular feature set and hyper-parameter setting*

1. Split the data into K (say 5 or 10) equal sized parts



# K-fold cross-validation

*Given a particular feature set and hyper-parameter setting*

1. Split the data into K (say 5 or 10) equal sized parts

2. Train a classifier on four parts and evaluate it on the fifth one



# K-fold cross-validation

*Given a particular feature set and hyper-parameter setting*

1. Split the data into K (say 5 or 10) equal sized parts

2. Train a classifier on four parts and evaluate it on the fifth one

3. Repeat this using each of the K parts as the *validation set*

| | | | | | |
|---|---|---|---|---|---|
| Part 1 | Part 2 | Part 3 | Part 4 | **Part 5** | $Accuracy_5$ |
| Part 1 | Part 2 | Part 3 | **Part 4** | Part 5 | $Accuracy_4$ |
| Part 1 | Part 2 | **Part 3** | Part 4 | Part 5 | $Accuracy_3$ |
| Part 1 | **Part 2** | Part 3 | Part 4 | Part 5 | $Accuracy_2$ |
| **Part 1** | Part 2 | Part 3 | Part 4 | Part 5 | $Accuracy_1$ |

# K-fold cross-validation

*Given a particular feature set and hyper-parameter setting*

1. Split the data into K (say 5 or 10) equal sized parts

2. Train a classifier on four parts and evaluate it on the fifth one

3. Repeat this using each of the K parts as the *validation set*

4. The quality of this feature set/hyper-parameter is the average of these K estimates

Performance = $(\text{accuracy}_1 + \text{accuracy}_2 + \text{accuracy}_3 + \text{accuracy}_4 + \text{accuracy}_5)/5$

# K-fold cross-validation

*Given a particular feature set and hyper-parameter setting*

1. Split the data into K (say 5 or 10) equal sized parts

2. Train a classifier on four parts and evaluate it on the fifth one

3. Repeat this using each of the K parts as the *validation set*

4. The quality of this feature set/hyper-parameter is the average of these K estimates

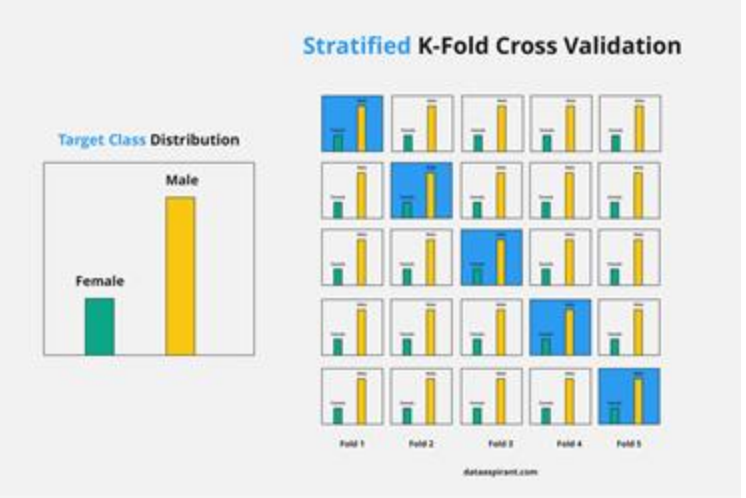
$$\text{Performance} = (\text{accuracy}_1 + \text{accuracy}_2 + \text{accuracy}_3 + \text{accuracy}_4 + \text{accuracy}_5)/5$$

5. Repeat for every feature set/hyper parameter choice

# Stratified K-Fold Cross-Validation

**A variation of k-fold cross-validation** where the data is split into k folds, but with the constraint that each fold maintains the same proportion of samples from each class as the entire dataset.

- useful when the **dataset is imbalanced** (e.g., one class is much more frequent than the other).
  - ensure each fold has a similar class distribution for fairer model evaluation.




Assume Classification problem, Having 3 class(A,B,C) to predict.

| Class | No_of_instance |
|---|---|
| A | 50 |
| B | 50 |
| C | 50 |

**KFold (K=5):**
Randomly splits data into 5 folds, where the number of instances per class may vary.

**StratifiedKFold:**
Splits data into 5 folds, ensuring each fold has an equal and proportional distribution of instances from each class A, B and C.

In general, stratified k-fold is recommended for classification tasks, especially when dealing with imbalanced datasets.

# Leave One Out Cross Validation (LOOCV)

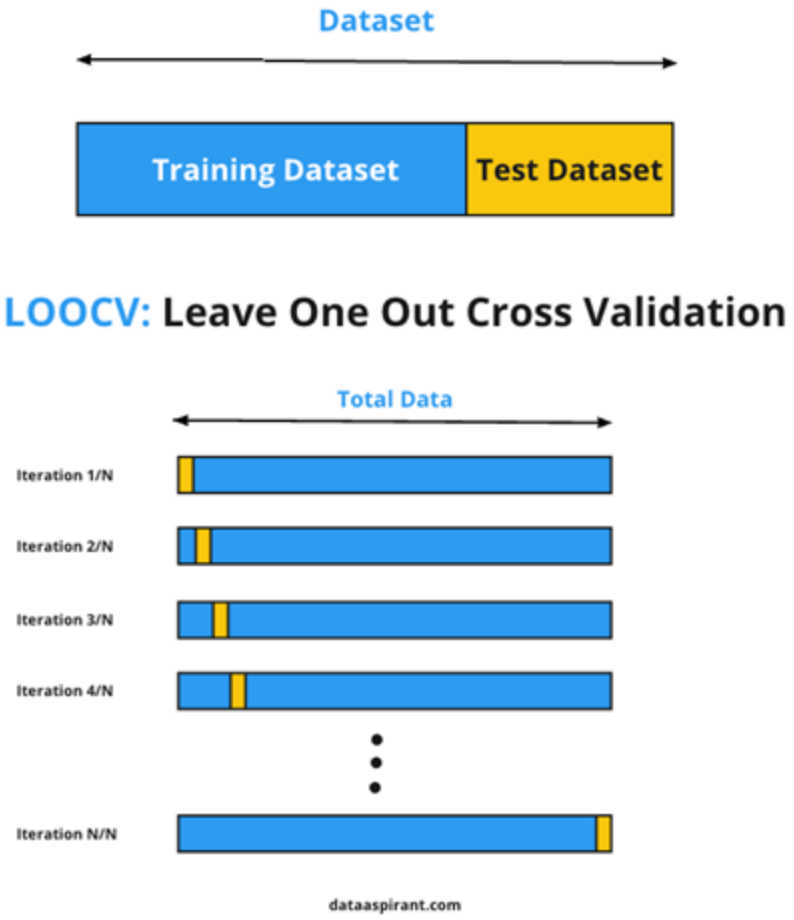
Estimates a model's performance by **training it on all data points except one**, and then using that **left-out point to test the model.**

When every ounce of training data counts

- Unlike K-Fold, there is no separate validation set.
- Each iteration focuses on evaluating the model's performance on the left-out observation.
- This process is repeated for every data point.

Particularly useful for small datasets.

**Drawbacks:** Computationally expensive and slower for larger datasets. However, it provides a highly reliable performance estimate.



# Leave One Out Cross Validation (LOOCV) Process

- **Split a dataset into a training set and a testing set**, using all but one observation as part of the training set.
  - Train the model on n-1 data points (where n is the total number of data points).
  - Leave one observation out from the training set (This is where the method gets the name “leave-one-out” cross-validation).
- This process is repeated 'n' times, with each data point being used as the test set once.



# ML Model Evaluation

*Using some evaluation metrics*

# Accuracy

The simplest way to check how well our model is doing is to look at it's **accuracy.**

$$accuracy = \frac{correct}{total}$$

100% accuracy is perfect, 0% accuracy is completely wrong.

# Example: Accuracy

Question: "If 90 out of 100 predictions are correct for spam detection in an email classification model, what is the model's accuracy?"

$$Accuracy = \frac{Number\ of\ Correct\ Predictions}{Total\ Number\ of\ Predictions}$$

= 90/100 * 100% = 0.9 * 100= 90%

**However, 'Accuracy' cannot always be used as the sole metric to evaluate model performance**

# Class Imbalance

Imagine a dataset with 90% of the negative class and 10% of the positive class--is 80% accuracy impressive here? What about 90%?

Class imbalance is a serious issue. Class imbalance happens when an overwhelming majority if your data is a single class.

# Situations With Class Imbalance

**Class Imbalance Issue:** Occurs when one class dominates the data, making one group significantly larger than others.

Examples of Imbalance:

- **Fraud:** Most transactions are not fraudulent.
- **Disease:** The majority don't have cancer.
- **Purchases:** Most people don't buy a specific product.

Common Scenario: Typically involves **a rare positive signal amidst negatives**, which is a prevalent occurrence.

# Limitation of “Accuracy” Metric

*“Accuracy is utterly useless if the class distribution in your data set is skewed”*

**Issue with Imbalanced Data:** Accuracy can mislead when one class dominates.

- A high accuracy may come from predicting the majority class, ignoring minority performance.

**Neglecting Error Types:** Accuracy treats all errors equally, not distinguishing between false positives and false negatives.

So more alternative evaluation metrics are needed. **Confusion Matrix**

# Classifier Evaluation Metrics: Confusion Matrix

A confusion matrix is a matrix that **summarizes the performance of a machine learning model on a set of test data.**

**Confusion Matrix:**

| Actual class\ Predicted class | C | ¬ C | Total |
|---|---|---|---|
| C | True Positives (TP) | False Negatives (FN) | P |
| ¬ C | False Positives (FP) | True Negatives (TN) | N |
| Total | P' | N' | P+N |

**True Positives (TP):** Positive tuples correctly labeled by classifier.

**True Negatives (TN):** Negative tuples correctly labeled by classifier.

**False Positives (FP):** Negative tuples incorrectly labeled as positive by classifier.

**False Negatives (FN):** Positive tuples incorrectly labeled as negative by classifier.

# Classifier Evaluation Metrics: Confusion Matrix

| Actual class\ Predicted class | C | ¬ C | Total |
|---|---|---|---|
| C | True Positives (TP) | False Negatives (FN) | P |
| ¬ C | False Positives (FP) | True Negatives (TN) | N |
| Total | P' | N' | P+N |

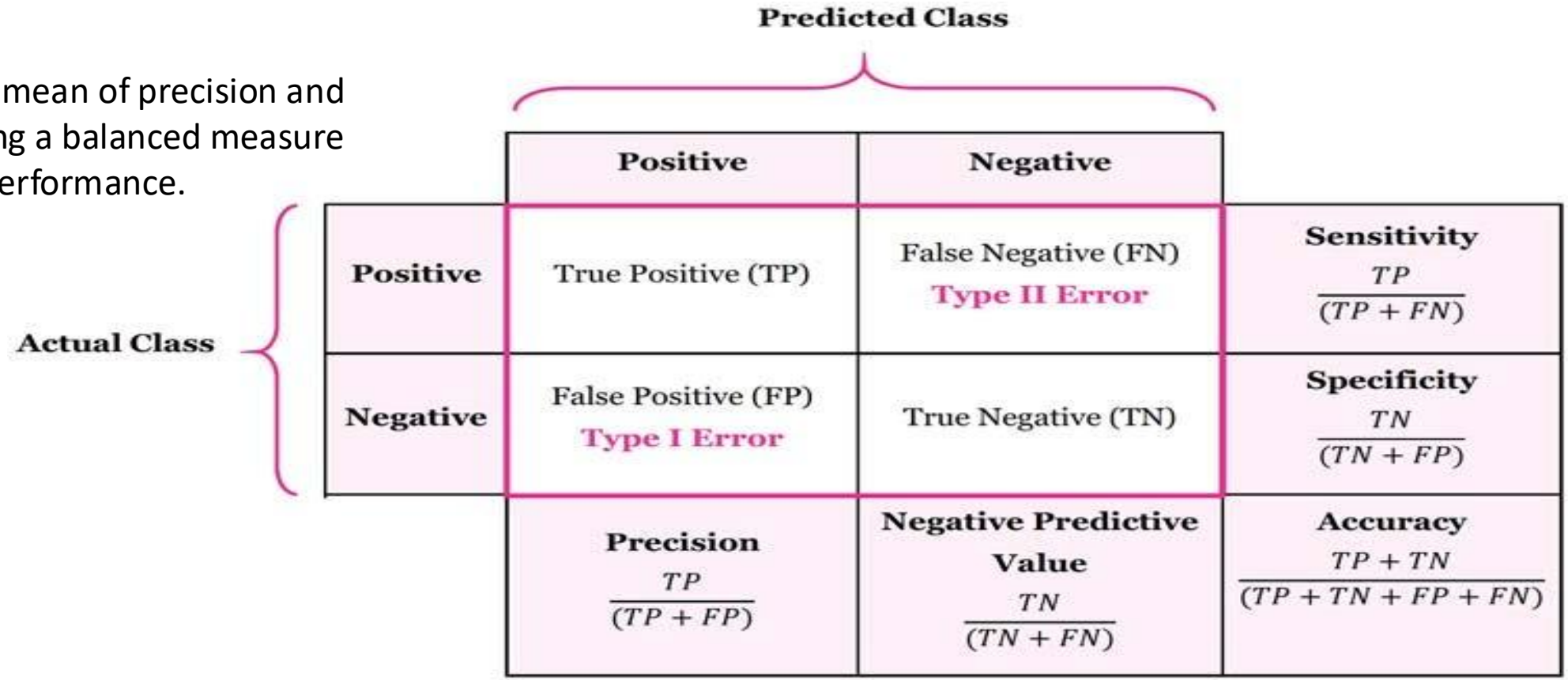
**Example of Confusion Matrix:**

| Actual class\Predicted class | buy_computer = yes | buy_computer = no | Total |
|---|---|---|---|
| buy_computer = yes | **6954** | **46** | 7000 |
| buy_computer = no | **412** | **2588** | 3000 |
| Total | 7366 | 2634 | 10000 |

# The confusion matrix allows to calculate various evaluation metrics

- Accuracy
- Precision
- Recall (Sensitivity or True Positive Rate)
- F1-Score

  The harmonic mean of precision and recall, providing a balanced measure of a model's performance.



# Classifier Evaluation Metrics:

## *Accuracy, Error Rate*

**Accuracy**, ***recognition rate***: percentage of test set tuples that are correctly classified

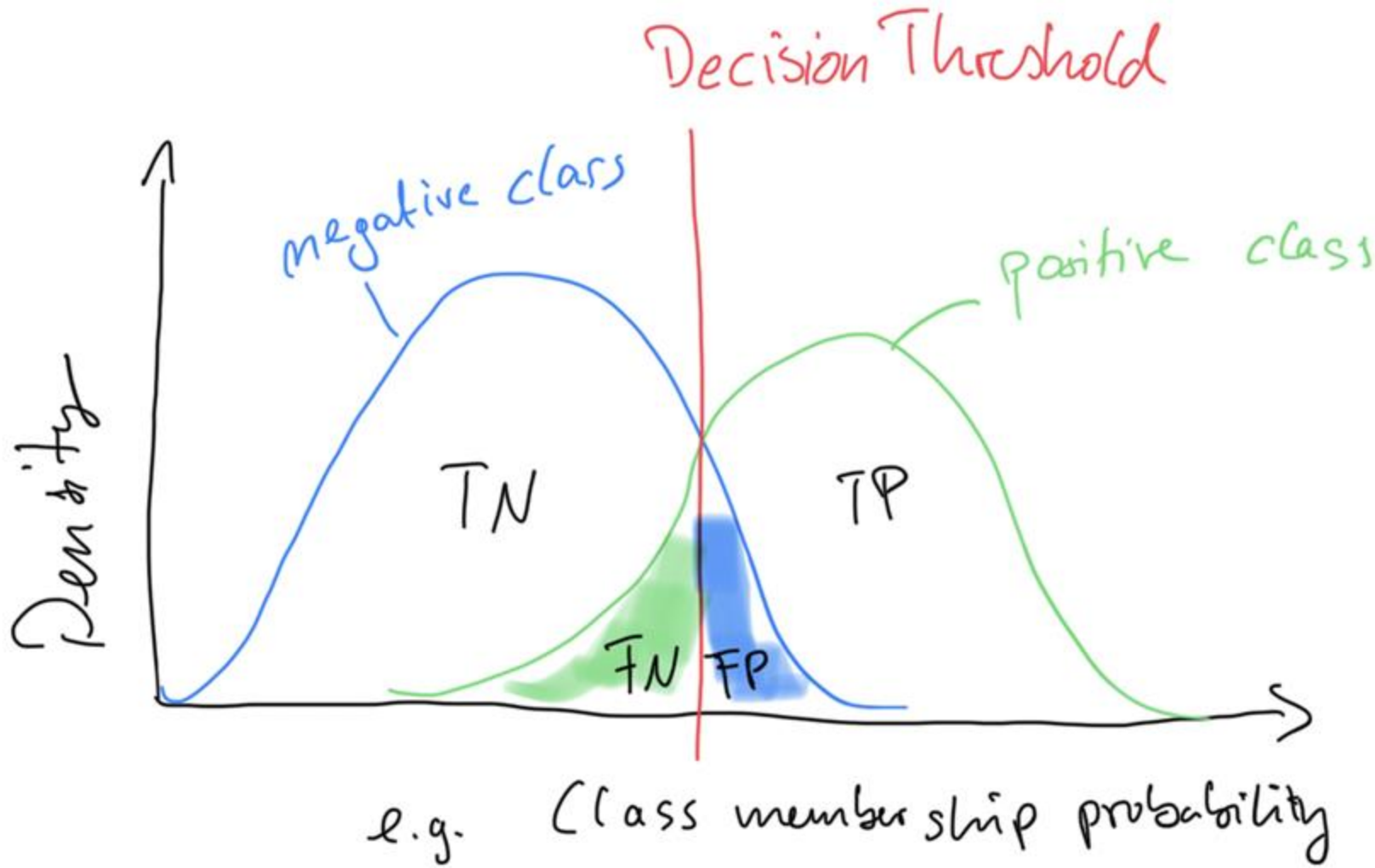
**Accuracy = (TP + TN) / (TP+FP+TN+FN)**

**Error Rate**, ***misclassification rate***: 1 – Accuracy

**Error Rate = (FP + FN) / (TP+FP+TN+FN)**

| Actual\ Predicted | C | ¬ C |
|---|---|---|
| C | TP | FN |
| ¬ C | FP | TN |

# False Positive Rate and False Negative Rate



# Precision

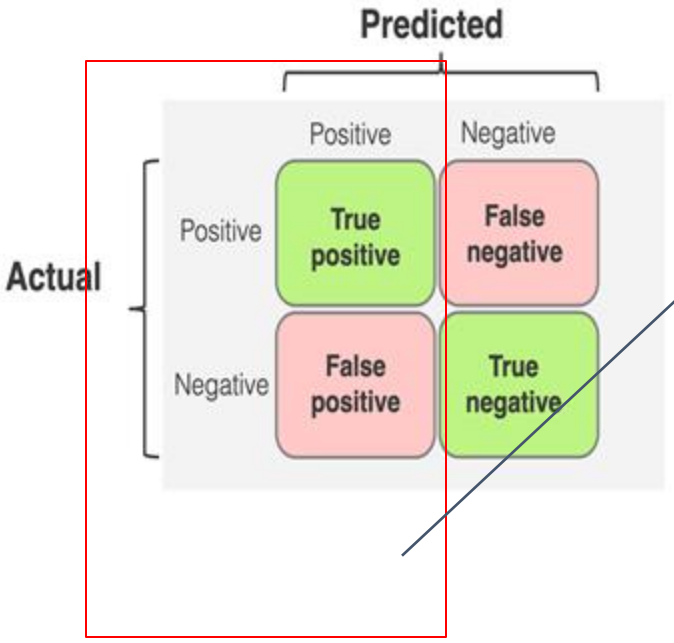
How much we are **precise (accurate)** in identifying positive cases

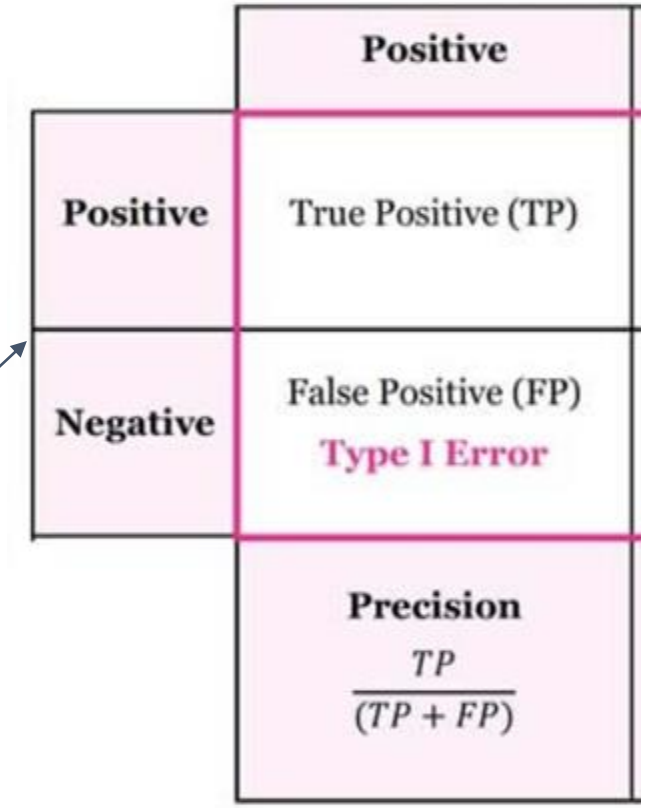
# Evaluation Metric: Precision

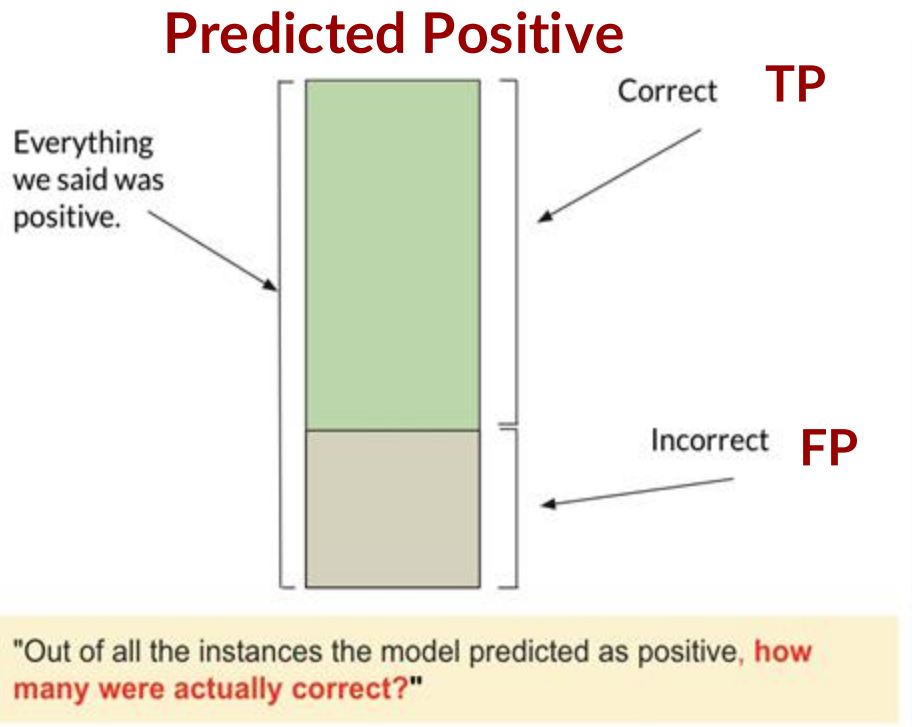
Precision is a metric that measures the **accuracy of positive predictions made by a model.**

Out of all positive predictions made, it **indicates the proportion that are truly positive.**

$$precision = \frac{tp}{tp + fp}$$



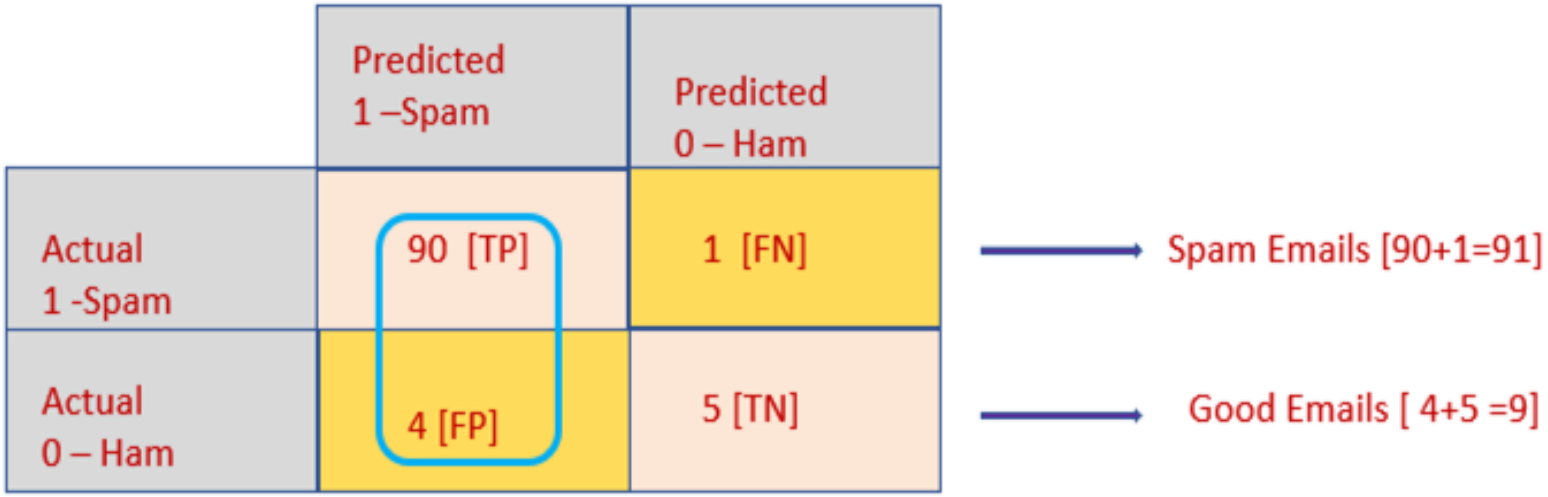




"Out of all the instances the model predicted as positive, how many were actually correct?"

# Example: Precision

**Interpretation:** Indicates the proportion of **true positive results among all positive predictions**.




$$\text{Precision} = \frac{90}{90+4} = 0.95$$

Precision = 95%

- Measured on a scale of 0 to 1 or as a percentage.
- **Higher precision is better**.
- value 1.0 indicates the model is always correct when predicting the target class and *never makes a mistake*.

# When to Use Precision

We use precision when **we only care about being correct about the things we identify as positive.**

- Google does not care if it turns away 1000 good engineers; Google just wants to make sure the ones it DOES hire are good

precision answers the question: how often the positive predictions are correct?

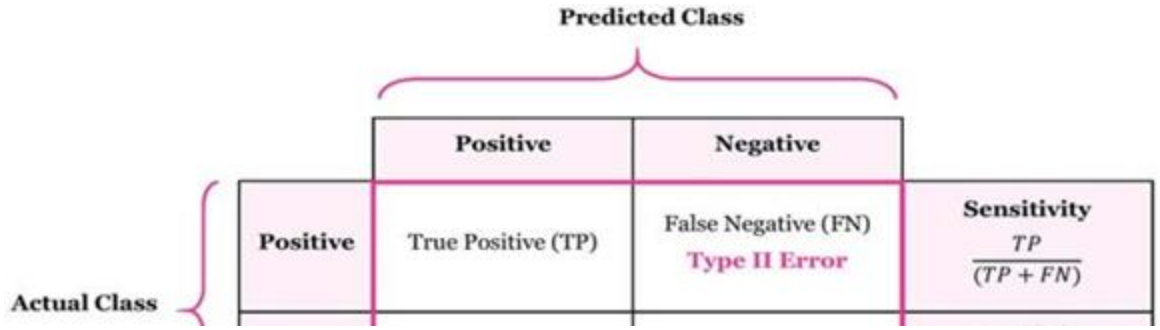
# Recall or Sensitivity

How much we are good at detecting/ identifying all actual positive cases?

# Recall (Sensitivity or True Positive Rate)

Recall is a measure of h**ow many actual positives your model is able to recall from the data.**

$$Recall = \frac{tp}{tp + fn}$$



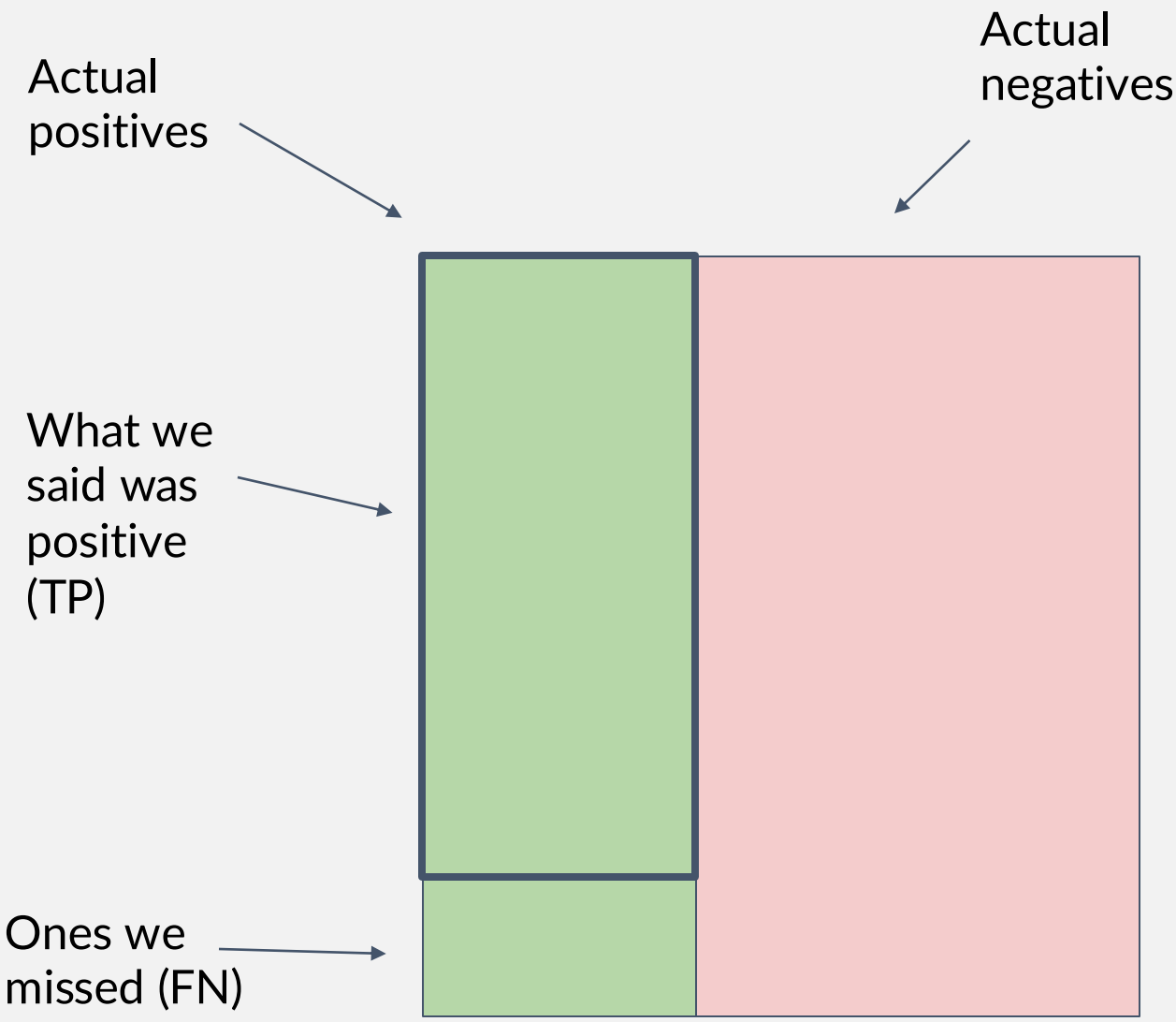

**Recall:** The proportion of true positives to the total actual positives (TP / (TP + FN)). It measures the model's ability to ***identify all positive instances***.

# Recall

Recall focus how many of our positive class that we ***missed (actually positive but we predicted as negative FN)***.

| | | Actual | |
|---|---|---|---|
| | | Positive | Negative |
| Predicted | Positive | TP | FP |
| | Negative | FN | TN |

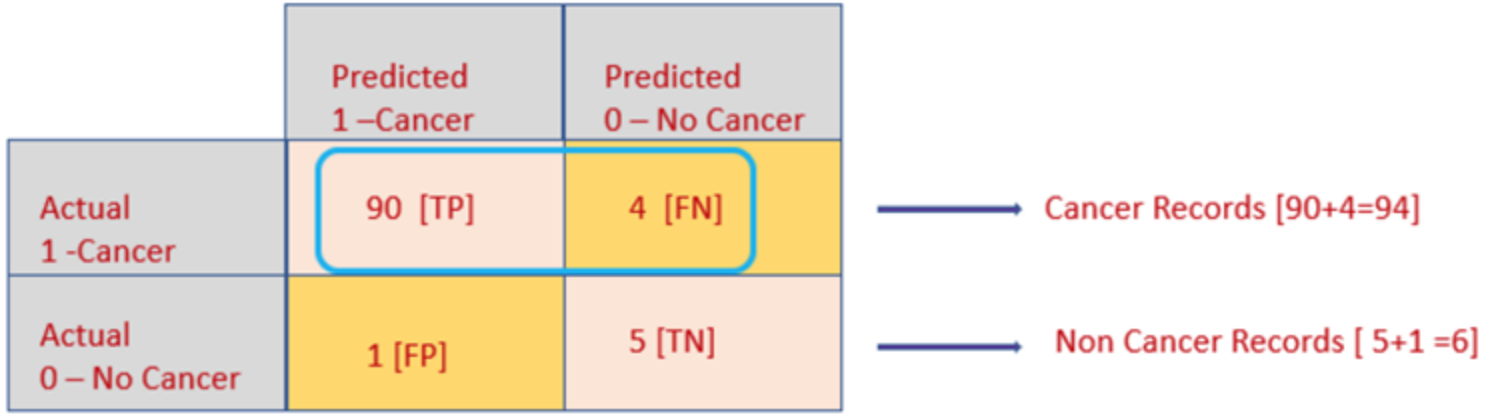
$$Recall = \frac{tp}{tp + fn}$$




"Out of all the **actual positive cases**, how many did the model correctly identify?"

# Example: Recall

## Recall




$$Recall = \frac{90}{90 + 4} = 0.95$$

Recall = 95%

- Measured on a scale of 0 to 1 or as a percentage.
- **Higher recall is preferable; it indicates fewer false negatives.**
- Crucial in scenarios where missing a positive instance is costly (e.g., disease detection, fraud detection).
- value of 1.0 means the model captures all instances of the target class and never misses it in predictions.

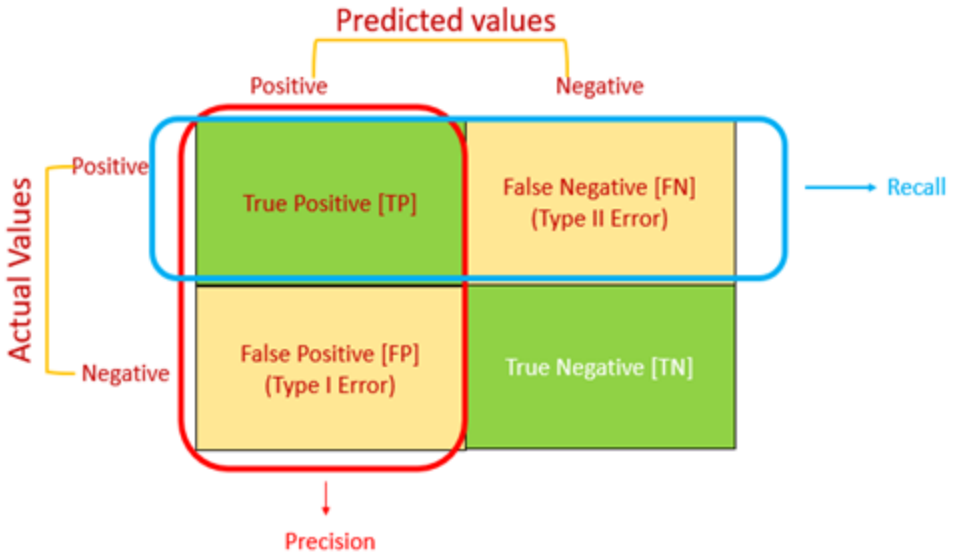
# When to use Recall:

We'd use recall when we want to make sure ***we don't miss anything***.

- For example identifying people contagious with a deadly super plague

# Precision vs. Recall

In the given scenarios, which metric (precision or recall) holds more practical importance or application?




- Testing for cancer
- Our legal system
- Fraud alerts
- Loans

Think about:
**Precision:** Aims **to be right** when it **says something is positive** (minimize false positives).
**Recall:** Aims to **not miss anything** that's **actually positive** (minimize false negatives).

# Precision vs. Recall

- **Testing for cancer**
  - Recall: Don't want to be wrong (all our cancer records should be predicted correctly), and we can always do further tests
- **Our legal system**
  - Depends ( *precision if aim to reduce false accusations or arrests; recall if priority is to capture all crimes, even if it requires extensive investigations*).
- **Fraud alerts**
  - Recall: Better to deny some good transactions than pay for the fraudulent ones
- **Loans**
  - Precision: We only want to loan money to people who will pay it back

# Evaluating Confidence in Predictions: What about our confidence?

## Incorporating Confidence into Model Evaluation

Confidence refers to how sure a model is about its prediction.

Machine learning algorithms often provide a **confidence score** (a probability, indicating the likelihood of a prediction being correct) for each prediction.

- Different confidence levels can lead to different action.
  - If a model predicts "yes" with 99% confidence and it's wrong, this should be penalized more than if the model only predicted with 60% confidence.
    - Precision and recall don't account for confidence, treating both predictions the same despite differing certainty.
    - One of the solution: Use **log or cross-entropy loss** - incorporates both accuracy and confidence

# What if Confidence Matters?

**Log Loss measures the accuracy of a model's predictions, specifically evaluating the confidence of predicted probabilities in classification tasks.**

Log Loss is a measure of accuracy that penalizes overconfidence by assigning higher penalties to confident but incorrect predictions, encouraging models to deliver both accurate and confident outputs.

- N =total number of data points
- M=outcomes or classes (usually 0 and 1).
- Xij→ Actual outcome for data point $i$ for class $j$
- Pij→ predicted probability that data point i belongs to class j.

$$\text{Log loss} = \frac{1}{N} \sum_{i=1}^{N} -\left(y_i * \log(p_i) + (1-y_i) * \log(1-p_i)\right)$$

**For Binary Classification**

$$LogLoss = \frac{-1}{N} \sum_{i=1}^{N} \sum_{j=1}^{M} x_{ij} * \ log(p_{ij})$$

**For multi-class classification where N is no. or rows and M is no. of classes**

A lower log loss indicates better model performance, with values closer to 0 being ideal.

Log loss helps us understand how well the model's confidence matches reality; making it a crucial metric for evaluating binary classification models.

# F1 Score

In some scenarios, we aim to minimize both false positives (FPs) and false negatives (FNs), maximizing both precision and recall.

**Example:**

When identifying harmful content:

- **High Precision:** Ensures flagged content is accurate.
- **High Recall:** Ensures no inappropriate content is overlooked.

**Challenge:** It's often impossible to maximize both metrics simultaneously due to their trade-off: *"Increasing precision typically decreases recall, and vice versa."*

**Solution: F1 Score**

# F1 Score

used when you seek a balance between precision and recall.

**The F1 Score is a metric that combines precision and recall into a single value, providing a balanced measure of a model's performance**

$$\text{F1 Score} = \frac{2}{\frac{1}{\text{Precision}} + \frac{1}{\text{Recall}}} = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

**Range:** F1 Score ranges from 0 to 1; the closer it is to 1, the better the model's performance.

**Addresses the trade-off** between precision and recall by equally weighting both metrics.

**Ideal for imbalanced datasets** where ensuring both low false positives and low false negatives is crucial, such as in medical diagnoses or fraud detection.

# Summary

**Precision:** Focuses on minimizing false positives in predictions.

**Recall:** Focuses on minimizing false negatives in predictions.

**Log Loss:** Evaluates the alignment between predicted probabilities and true class probabilities, aiming for lower values closer to 0.

**F1 Score:** Balances both precision and recall in a single metric, useful for scenarios where false positives and false negatives are equally important.